

AI Assisted Coding ASSIGNMENT-9.5

Name: K. Sai Krishna Reddy

HT.No: 2303A52305

Batch: 45

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):  
    return text[::-1]
```

Task:

1. Write documentation in:
 - o (a) Docstring
 - o (b) Inline comments
 - o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

PROMPT:

Generate documentation for the following Python function in three styles:

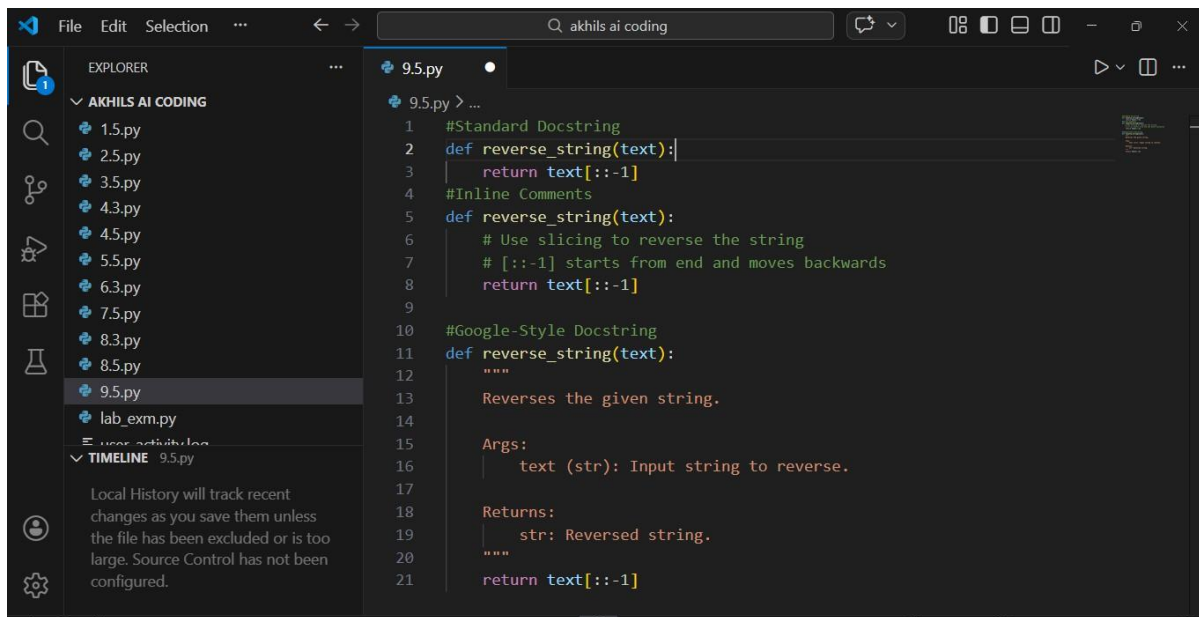
- 1) Standard docstring
- 2) Inline comments
- 3) Google-style docstring

Function:

```
def reverse_string(text):  
    return text[::-1]
```

Also compare the three styles and recommend the best style for a utility-based string library.

OUTPUT:



Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):
```

```
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

PROMPT:

Document the following security-related function using:

- 1) Standard docstring
- 2) Inline comments
- 3) Google-style documentation

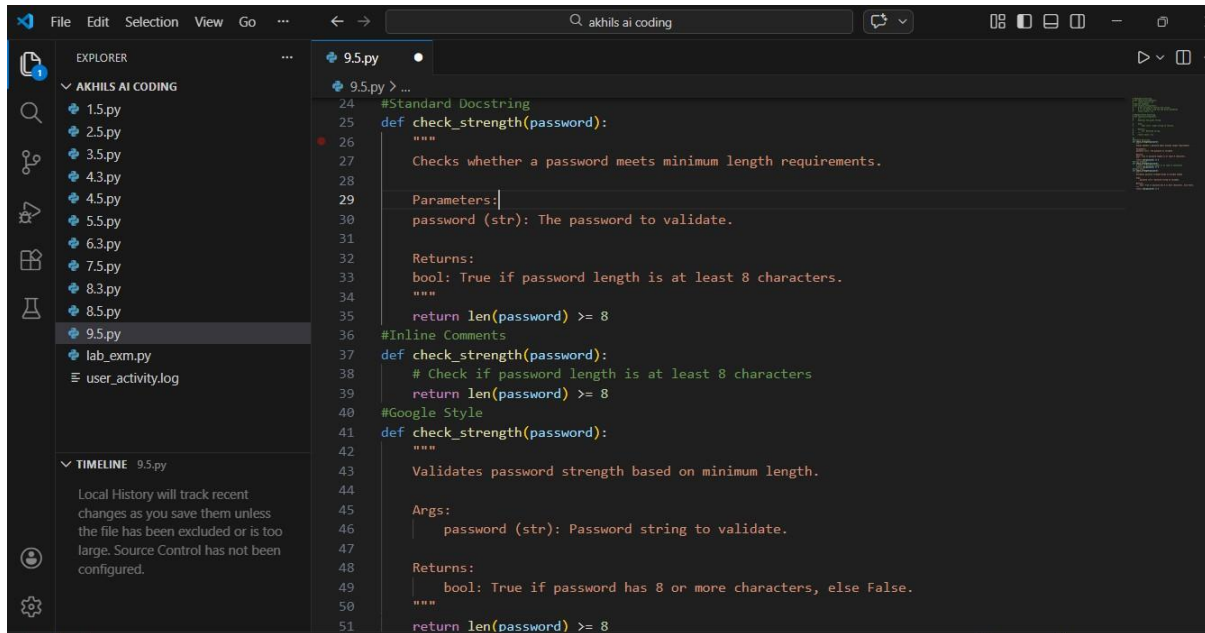
Function:

```
def check_strength(password):
```

```
    return len(password) >= 8
```

Compare documentation styles for security-related code and recommend the best one.

OUTPUT:



```
24 #Standard Docstring
25 def check_strength(password):
26     """
27     Checks whether a password meets minimum length requirements.
28
29     Parameters:
30     password (str): The password to validate.
31
32     Returns:
33     bool: True if password length is at least 8 characters.
34     """
35     return len(password) >= 8
36 #Inline Comments
37 def check_strength(password):
38     # Check if password length is at least 8 characters
39     return len(password) >= 8
40 #Google Style
41 def check_strength(password):
42     """
43     Validates password strength based on minimum length.
44
45     Args:
46     password (str): Password string to validate.
47
48     Returns:
49     bool: True if password has 8 or more characters, else False.
50     """
51     return len(password) >= 8
```

Problem 3: Math Utilities Module

Task:

1. Create a module math_utils.py with functions:
 - o square(n)
 - o cube(n)
 - o factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

PROMPT:

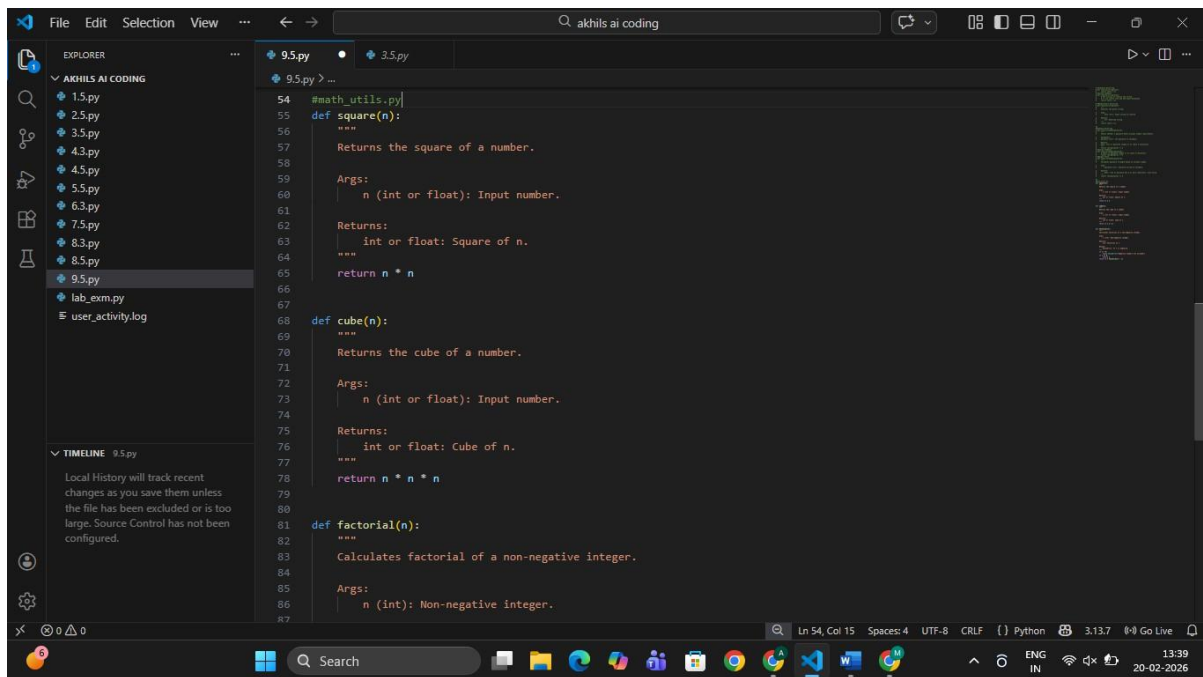
Create a Python module named math_utils.py with functions:

- square(n)
- cube(n)
- factorial(n)

Add Google-style docstrings.

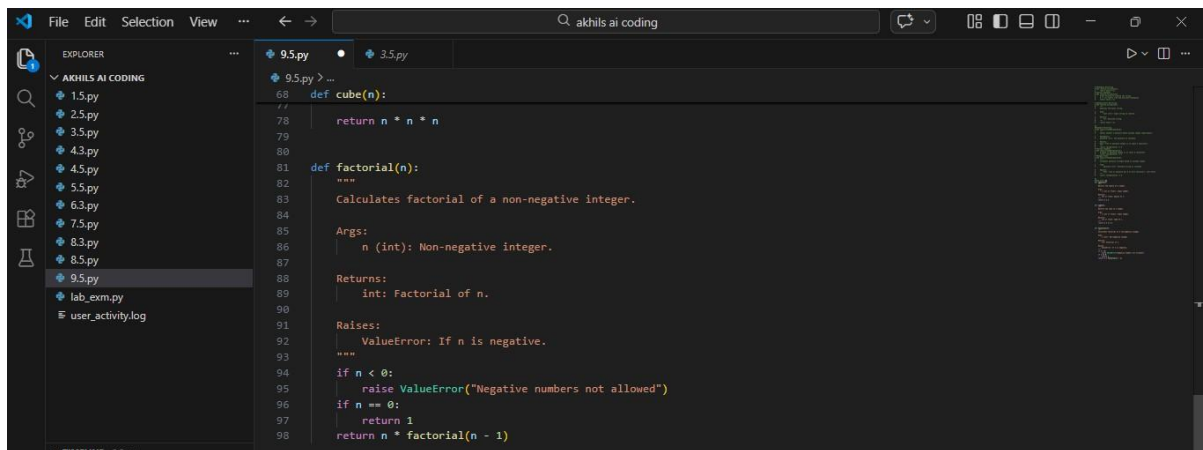
Generate documentation using Sphinx and export it as HTML.

OUTPUT:



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays a project named 'AKHILS AI CODING' with several Python files. The file '9.5.py' is selected and open in the editor. The code in '9.5.py' defines three functions: `square(n)`, `cube(n)`, and `factorial(n)`. Each function has a docstring describing its purpose and arguments. The `square` function returns the square of a number, `cube` returns the cube, and `factorial` calculates the factorial of a non-negative integer. The status bar at the bottom indicates the current line and column (Ln 54, Col 15) and the file encoding (UTF-8).

```
54 #math_utils.py
55 def square(n):
56     """
57     Returns the square of a number.
58
59     Args:
60         n (int or float): Input number.
61
62     Returns:
63         int or float: Square of n.
64     """
65     return n * n
66
67
68 def cube(n):
69     """
70     Returns the cube of a number.
71
72     Args:
73         n (int or float): Input number.
74
75     Returns:
76         int or float: Cube of n.
77     """
78     return n * n * n
79
80
81 def factorial(n):
82     """
83     Calculates factorial of a non-negative integer.
84
85     Args:
86         n (int): Non-negative integer.
87
```



This screenshot shows the same VS Code editor interface, but the code in '9.5.py' is different. It now defines two functions: `cube(n)` and `factorial(n)`. The `cube` function is a simple multiplication of `n` by itself three times. The `factorial` function calculates the factorial of a non-negative integer, including a check for negative values and a base case for 0. The status bar at the bottom indicates the current line and column (Ln 54, Col 15) and the file encoding (UTF-8).

```
68 def cube(n):
69     """
70     Returns the cube of a number.
71
72     Args:
73         n (int or float): Input number.
74
75     Returns:
76         int or float: Cube of n.
77     """
78     return n * n * n
79
80
81 def factorial(n):
82     """
83     Calculates factorial of a non-negative integer.
84
85     Args:
86         n (int): Non-negative integer.
87
88     Returns:
89         int: Factorial of n.
90
91     Raises:
92         ValueError: If n is negative.
93     """
94     if n < 0:
95         raise ValueError("Negative numbers not allowed")
96     if n == 0:
97         return 1
98     return n * factorial(n - 1)
```

Problem 4: Attendance Management Module

Task:

1. Create a module `attendance.py` with functions:
 - o `mark_present(student)`
 - o `mark_absent(student)`
 - o `get_attendance(student)`
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

PROMPT:

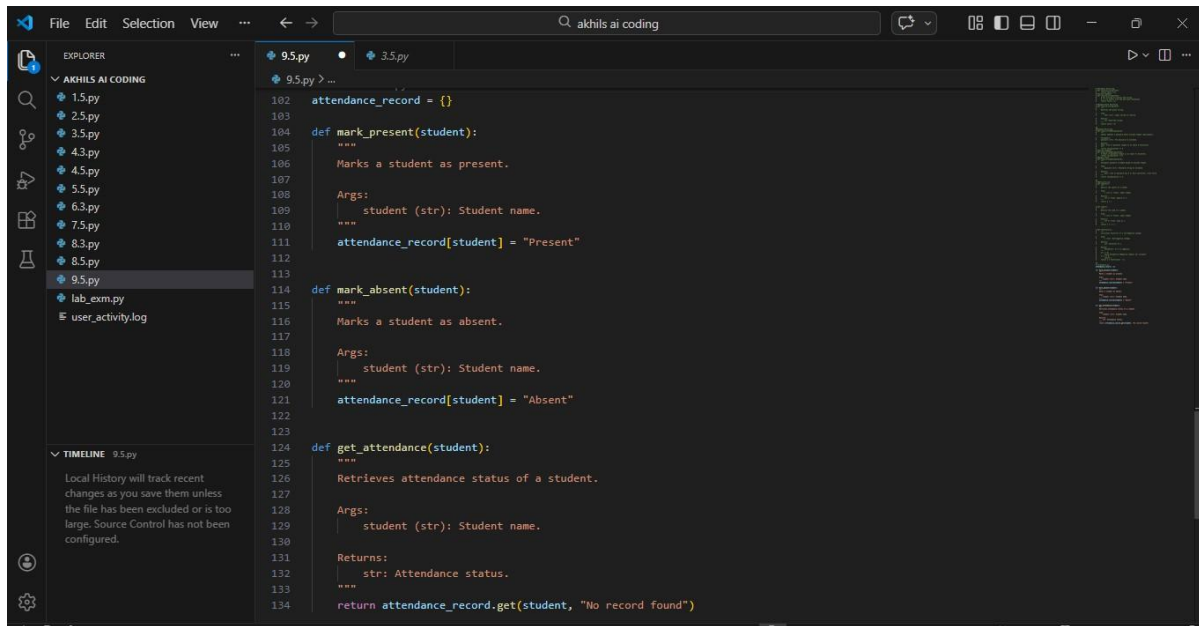
Create `attendance.py` module with:

- mark_present(student)
- mark_absent(student)
- get_attendance(student)

Add Google-style docstrings.

Generate documentation and show how to view in terminal and browser.

OUTPUT:



Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

PROMPT:

Document the following function in:

- 1) Standard docstring

2) Inline comments

3) Google-style documentation

Function:

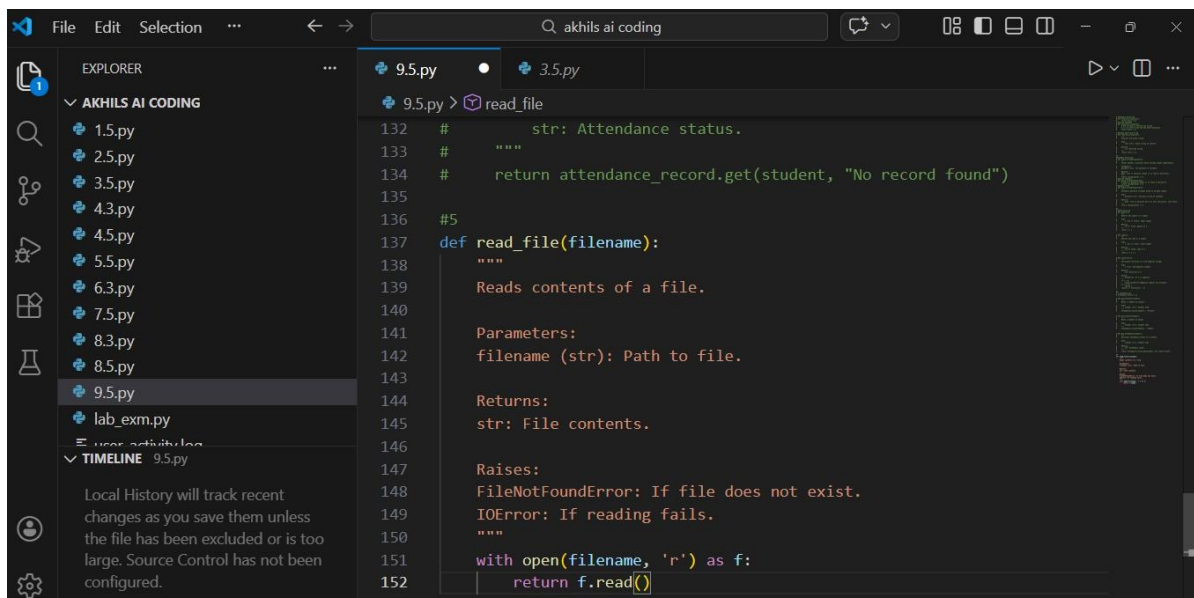
```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

Identify which style best explains exception handling and justify.

OUTPUT:



```
132 #         str: Attendance status.
133 #         """
134 #         return attendance_record.get(student, "No record found")
135
136 #5
137 def read_file(filename):
138     """
139     Reads contents of a file.
140
141     Parameters:
142     filename (str): Path to file.
143
144     Returns:
145     str: File contents.
146
147     Raises:
148     FileNotFoundError: If file does not exist.
149     IOError: If reading fails.
150     """
151     with open(filename, 'r') as f:
152         return f.read()
```